Next Generation of Salesforce Application Lifecycle Management (Pilot)

Salesforce, Winter ’26

Last updated: November 7, 2025

© Copyright 2000–2025 Salesforce, Inc. All rights reserved. Salesforce is a registered trademark of Salesforce, Inc., as are other names and marks. Other marks appearing herein may be trademarks of their respective owners.

Important: DevOps Center, DX Inspector for DevOps Center (Pilot), Metadata Deployment, and DevOps Center MCP Tools are pilot or beta services that are subject to the Beta Services Terms at Agreements - Salesforce.com or a written Unified Pilot Agreement if executed by Customer, and applicable terms in the Product Terms Directory. Use of this pilot or beta service is at the Customer's sole discretion.

Important: Data Deployment is available as a developer preview. Data Deployment isn't generally available unless or until Salesforce announces its general availability in documentation or in press releases or public statements. All commands, parameters, and other features are subject to change or deprecation at any time, with or without notice. Don't implement functionality developed with these commands or tools.

CONTENTS

Salesforce Application Lifecycle Management (ALM) 5
Salesforce ALM Features 5
Prerequisites 5
Salesforce ALM Journey 7
DevOps Center (Pilot) 9
Enhancements Over the Current DevOps Center 9
Considerations to Set Up DevOps Center 10
Get Started with the DevOps Center Setup 10
Add Team Members as Users in the DevOps Center Hub Org 11
Assign DevOps Center Permission Sets 11
Set Up Your Source Control 11
Add a Team Member to the Project Repository 11
Join a Project Repository as a Collaborator 12
Create a DevOps Center Project 12
Build Your Pipeline 13
Create and Assign Work Items 15
Next Steps 16
Integrate Agentforce Vibes IDE with DevOps Center 17
Get Started with Salesforce DX MCP Server 17
Build Using DX Inspector 19
Manage Changes 19
Add Metadata Manually 20
Commit Changes to Source Repository 21
Create a Change Review 22
Review and Promote Work Items in DevOps Center 23
Review Changes with Team Members 23
Promote Work Items 23
Deploy Metadata and Data from DX Inspector 25
DevOps Center MCP Tools (Pilot) 26
Benefits of DevOps Center MCP Tools 26
Use Cases 26
End-to-End Work Item Lifecycle Management 26
Resolve Merge Conflicts 28
Resolve Conflicts from the DevOps Center UI (Low-Code Admins) 30
Tool Reference 30
Work Item 30
Pull Request 31
Pipeline and Deployment 32
Conflict Resolution 32
Glossary 33

Limitations **33**
Tips for Writing Prompts for Salesforce DX MCP Server 34

Salesforce Application Lifecycle Management (ALM)

Salesforce ALM provides a unified, AI-powered experience to build and deploy Salesforce apps with confidence. With an interconnected ecosystem of agentic products, you can accelerate innovation and deliver high-quality apps while adhering to ALM best practices.

Here are the key benefits of Salesforce ALM:

- Streamlined and efficient ALM. The new modular DevOps Center architecture is integrated with the Salesforce Platform. This integration provides capabilities, such as API integrations and UI extensibility. It also simplifies the DevOps Center setup by removing the need for package downloads and installs.
- Flexible API-first architecture. All DevOps Center features are accessible via APIs, and these APIs support the integration of your workflows with DevOps Center.
- Unified development and deployment. With the connected experience, you can build, test, and deploy apps from a single source of truth.
- Natural language prompts for DevOps tasks. This agentic experience, powered by DevOps Center MCP (Model Context Protocol) tools in Salesforce DX MCP Server, supports end-to-end lifecycle management and conflict resolution.
- Comprehensive change tracking. Easily track and view changes across all your projects, apps, and deployments to ensure complete visibility.
- Single-orchestration deployment. Deploy apps, including both metadata and data from DX Inspector. This new deployment feature is an evolution of change sets.

Salesforce ALM Features

Understanding the core features and products is essential before beginning your ALM journey. Here's an overview of the products or features listed in this document.

- DevOps Center
- DX Inspector
  - Metadata and Data Deployment from DX Inspector
- DevOps Center MCP Tools in Salesforce DX MCP Server

Note: This document references the new, enhanced version of the DevOps Center, currently in pilot.

Prerequisites

Before you start your ALM journey with Salesforce, let's look at the prerequisites and user permissions for each feature.

USER PERMISSIONS

Note: Individual user permissions for each feature and product are detailed in their respective sections.

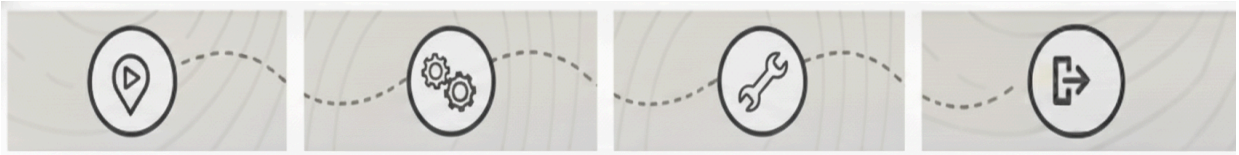
- To access DX Inspector: Customize Application
- To deploy metadata and data to a target org: Customize Application in the source and target orgs.

We recommend completing or installing the prerequisites in advance.

Product/Feature	Prerequisites
DevOps Center	• A GitHub account. • Access to the pilot trial org. Use the credentials received in your email.
DevOps Center MCP tools in Salesforce DX MCP Server	• An MCP-enabled IDE, such as Agentforce Vibes IDE or VS Code. • A user-facing MCP client, such as Agentforce Vibes Extension.
Metadata deployment	• An authorized target org with required permissions. • See Plan and Prepare for Deployment.
Data deployment	

Salesforce ALM Journey

Here's a high-level overview of your ALM journey. Start by setting up DevOps Center, then build your apps using DX Inspector, and finally, deploy your changes with either DevOps Center or metadata and data deployment feature.



Set Up DevOps Center	Start Working in DevOps Center	Build with DX Inspector	Promote and Deploy Changes
1. Set up DevOps Center in Hub org 2. Set up your source control 3. Create a DevOps Center project 4. Build your pipeline 5. *Integrate Agentforce Vibes IDE with DevOps Center 6. *Set up Salesforce DX MCP server * (You can complete these tasks later.)	1. Create a work item 2. Move your work item to **In Progress** state	1. Develop your app or make changes **With DevOps Center Connection** 2. Commit changes 3. Create a change review	1. Review and approve the changes 2. Promote changes through the pipeline
		Without DevOps Center Connection Use metadata and data deployment feature to deploy changes to a target org	

Task 1: Set up DevOps Center

DevOps Center is central to your ALM journey. If you're a new user, follow these steps to get started.

1. Access and set up DevOps Center in Hub org. See Add Team Members as Users in the DevOps Center Org.
2. Set up your source control. See Set Up Your Source Control.
3. Create a DevOps Center project. See Create a DevOps Center Project.

4. Build your DevOps Center pipeline. See Build Your Pipeline.
5. Create a DevOps Center work item. See Create and Assign a Work Item.
6. (Optional) Integrate Salesforce Agentforce Vibes IDE. See (Optional) Integrate Agentforce Vibes IDE with DevOps Center.
7. Set up Salesforce DX MCP Server. See Set Up the Salesforce DX MCP Server.

   Interested in DevOps Center MCP tools in Salesforce DX MCP Server?
   You can use DevOps Center MCP tools to manage your DevOps Center tasks with natural language prompts. For example, state what you want to do, such as "*Create a new feature branch for WI-0042*." See DevOps Center MCP Tools in DX MCP Server.

Task 2: Work in your development org

1. Build apps, or make changes directly in your development org.
   Your changes are automatically tracked via source tracking and are listed on the Change Management page.
2. Commit or deploy changes.
   After making changes, you have these options for committing or deploying them:
   - With a DevOps Center connection:
     i. Commit changes: Associate your changes with a work item and commit them to the project's source control repository. See Commit Changes to Source Repository.
     ii. Start review: Return to the Change Management page in DX Inspector, and start a review process. See Create a Change Review.
     iii. Team collaboration: After your team member reviews and approves the work item, the work item moves into `Ready to Promote` status. See Review Changes with Team Members.
     iv. Promote work items: When ready, promote the work items to the next stage in your pipeline. See Promote Work Items.

     Let the DevOps Center MCP tools in Salesforce DX MCP Server manage the promotion for you. For example, use the prompt, "*Promote DevOps Center WI-0042 to the UAT stage.*" Salesforce DX MCP Server runs all the necessary steps for pipeline deployment.

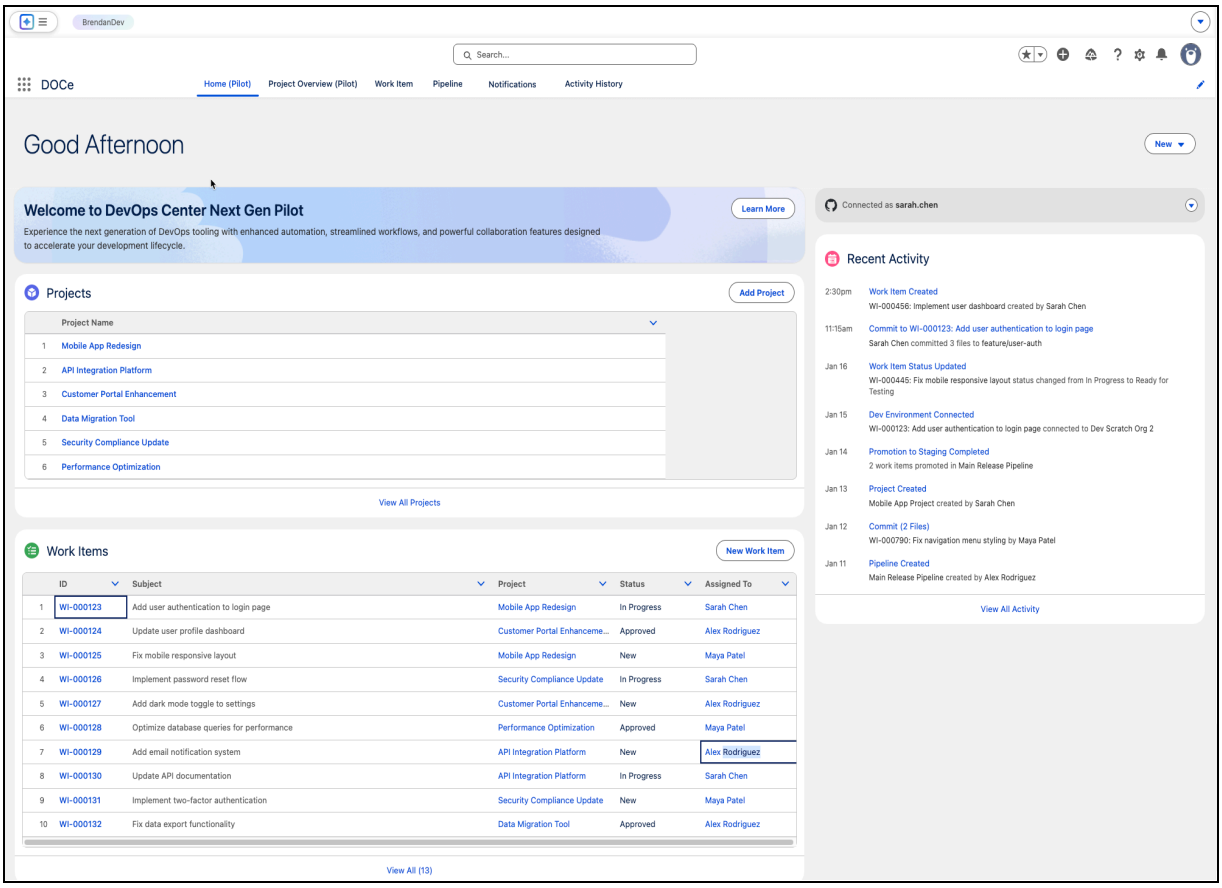
     In case of merge conflicts, instead of resolving conflicts manually, you can use DevOps Center MCP tools in Salesforce DX MCP Server to simplify the process. Describe the outcome in natural language prompts, and Salesforce DX MCP Server analyzes and resolves the merge conflicts for you. See Resolve Merge Conflicts.
   - Without a DevOps Center connection: For solo Salesforce admins or smaller teams, move metadata and data together for direct deployment to a target org. See Deploy Metadata and Data from DX Inspector.

DevOps Center (Pilot)

AI-powered, next-generation DevOps Center is built on principles similar to the existing DevOps Center and supports modern DevOps best practices through a click-based user interface (UI). It serves as a central hub for development and tracks all changes in a source control system. This creates a single source of truth and improves collaboration among developers, admins, release managers, and other stakeholders.

DevOps Center uses Salesforce DX MCP Server and DevOps Center MCP tools to provide AI-based recommendations for end-to-end work item management and conflict resolution.

DevOps Center Home Page



Enhancements Over the Current DevOps Center

While the current DevOps Center provides essential capabilities for pipeline, change, and release management, DevOps Center (Pilot) introduces these new capabilities:

- **AI-powered assistance and resolution**: An agentic DevOps experience using DevOps Center MCP tools in Salesforce DX MCP Server. Using natural language prompts, you can resolve complex DevOps issues, such as merge conflicts.

- **Ease of use**: DevOps Center doesn't require any package downloads and installs. After DevOps Center is enabled in your DevOps Center Hub org, you can instantly start using it.
- **Modern and extensible UI:** DevOps Center has a new, intuitive, and extensible UI built on Salesforce Lightning Design System (SLDS). This allows teams to customize workflows and integrate tools that match their specific governance models.
- **Flexible environment support:** Teams can build, track, and commit changes from non-source-tracked development environments. This is ideal for workflows with complex data configuration where source tracking isn't enabled.
- **Integrated metadata tracking**: DevOps Center integrates with DX Inspector to streamline tracking, committing, and reviewing metadata changes directly from your development orgs. This integration provides greater visibility and control over what's being promoted through your pipeline.

Considerations to Set Up DevOps Center

For the pilot program, a Salesforce representative sets up DevOps Center in your trial org. This org, where DevOps Center is enabled, is called the DevOps Center Hub org. You can access DevOps Center from the App Launcher, then connect it to any environment that you use for development, testing, and final release. Before you set up DevOps Center, review these considerations:

- To use DevOps Center, you must be a system administrator user. See Add a System Administrator to Your Dev Hub Org. For the pilot program, users with this profile can perform all DevOps Center tasks.
- Don't set up DevOps Center in a sandbox. Sandboxes can be connected to your pipeline as development or testing environments, but the DevOps Center Hub org itself can't be a sandbox.
- Connect your development orgs, pipeline environments, and release orgs from within DevOps Center.
- Use GitHub.com cloud-based plans, including GitHub Enterprise Cloud, for source control.

Get Started with the DevOps Center Setup

Before you start setting up DevOps Center, here's an overview of the process:

1. Add team members as users in the DevOps Center Hub org.
2. Assign system administrator profile to your team members.
3. Set up source control and add team members as collaborators.
4. Create a DevOps Center project.
5. Plan, build, and activate your pipeline.
6. Create and assign project work items.
7. (Optional) Integrate Agentforce Vibes IDE.
8. (Optional) Set up Salesforce DX MCP Server.

Add Team Members as Users in the DevOps Center Hub Org

Add any team members who aren't already users of the DevOps Center Hub org.

1. Log in to the DevOps Center Hub org.
2. From Setup, enter Users in the `Quick Find box`, then select **Users**.
3. Click **New User** or **Add Multiple Users**.
4. Select the appropriate license type and profile based on the user's role.
5. Select **Generate passwords and notify user via email**.
6. Save your changes.

Team members receive an invitation immediately. We recommend asking them to wait for your confirmation before logging in. Access to DevOps Center is limited until the setup is complete.

Assign DevOps Center Permission Sets

Assign system administrator profile to all your team members working on your projects in DevOps Center. Users with this profile can configure projects, build pipelines, and promote changes through the pipeline.

1. From Setup, enter **Permission Sets** in the `Quick Find box`, then select Permission Sets.
2. Select the **DevOps Center** permission set.
3. Click **Manage Assignments** and then **Add Assignments**.
4. Select the checkboxes next to the names of the users you want assigned to the permission set, and click **Assign**.
5. Click **Done**.

Set Up Your Source Control

Each DevOps Center pipeline requires its own GitHub repository to store project files and act as the single source of truth for your team. For the pilot program, DevOps Center supports only GitHub and uses GitHub REST API and OAuth 2.0 for secure communication and access.

Structure the repository as a Salesforce DX project, which is indicated by the presence of a `sfdx-project.json` file in the root directory. You can create the required repository in these ways:

- DevOps Center automatically creates it when you start a new pipeline.
- Use an existing repository or create one manually from the `dx-empty` template.

Add a Team Member to the Project Repository

If you're a team manager or Salesforce admin, complete these steps:

1. Ask each team member to create a GitHub account (if they don't have one already) and to send you their GitHub username.
2. Log in to GitHub and update the project repository settings for access to invite the team member as a collaborator.

3. Confirm with your team that they have received an email invitation from GitHub and have accepted it.

Join a Project Repository as a Collaborator

If you're a team member, complete these steps:

1. Create a GitHub account if you don't have one.
2. Send your GitHub username to the team manager or Salesforce admin who's setting up DevOps Center.
3. Check your email for an invitation to collaborate on the repository and then accept it.

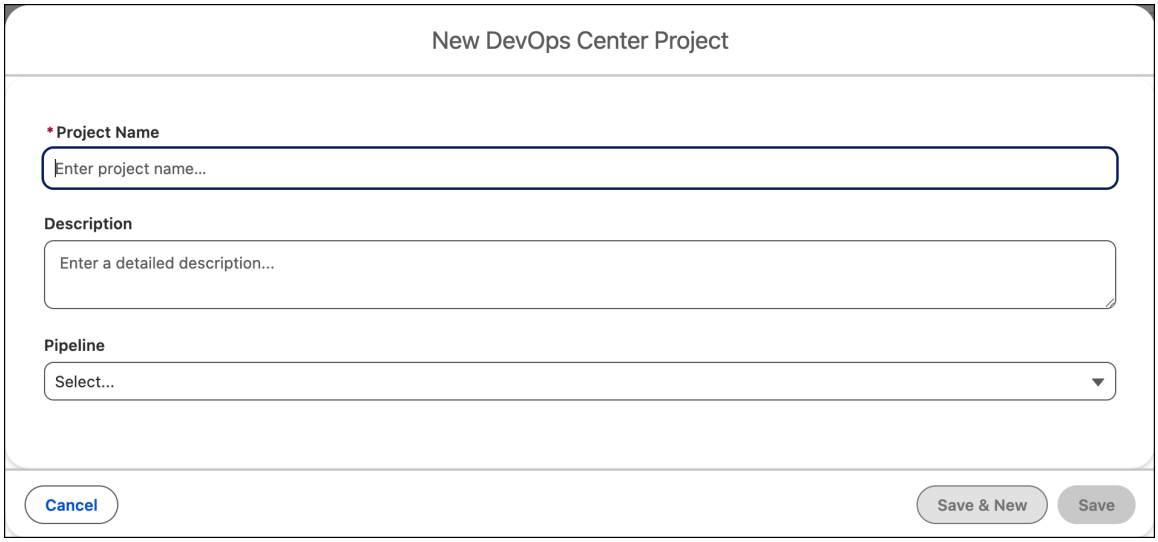
   If you haven't received an email invitation, contact the person who set up DevOps Center.

Create a DevOps Center Project

A DevOps Center project is the primary workspace where your team collaborates to manage changes for an application or customization. For the pilot program, each project is associated with a single pipeline.

1. From the DevOps Projects tab, click **New**.

New DevOps Center Project Window



2. Enter a unique name and description to identify the purpose of the project.
3. Select a pipeline to associate with this project.

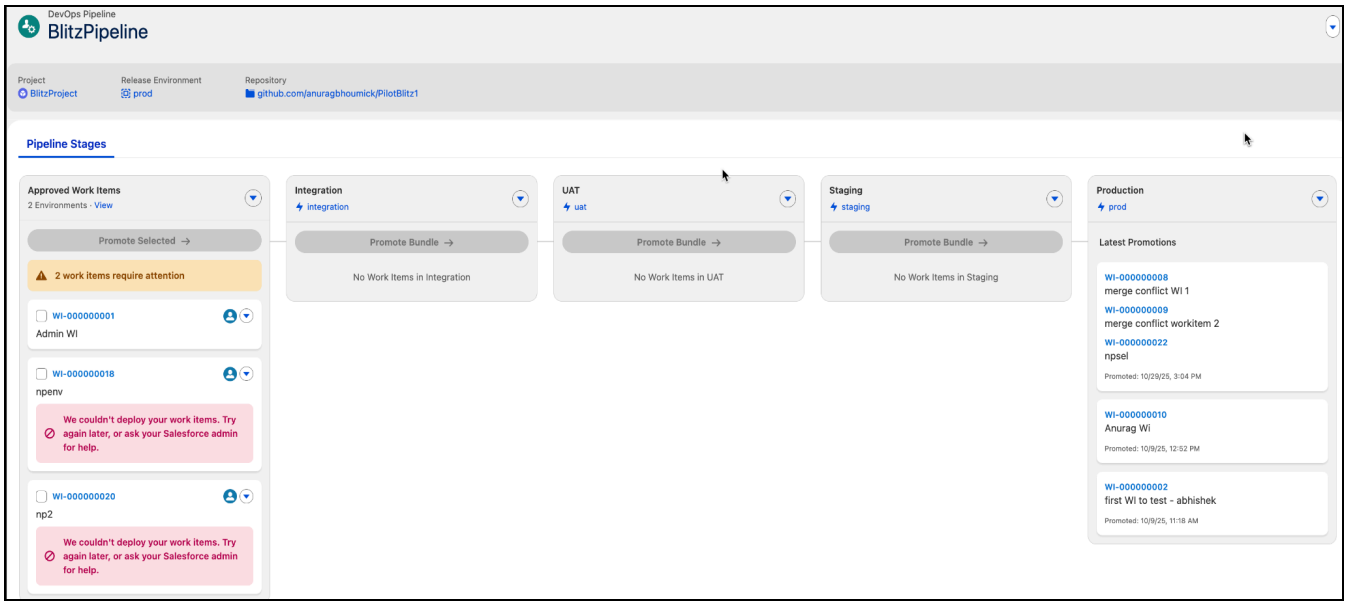
Note: If you haven't created and activated a pipeline, you can do so later. See Build Your Pipeline.

4. Save your project.

Build Your Pipeline

A pipeline defines the sequence of stages that work items progress through as they go through the release lifecycle from development to production (or some other final release stage). For example, Dev → Integration → UAT → Staging → Production. The pipeline consists of pipeline stages. Each pipeline stage corresponds to an environment (currently a Salesforce org) and a branch in your source control repository.

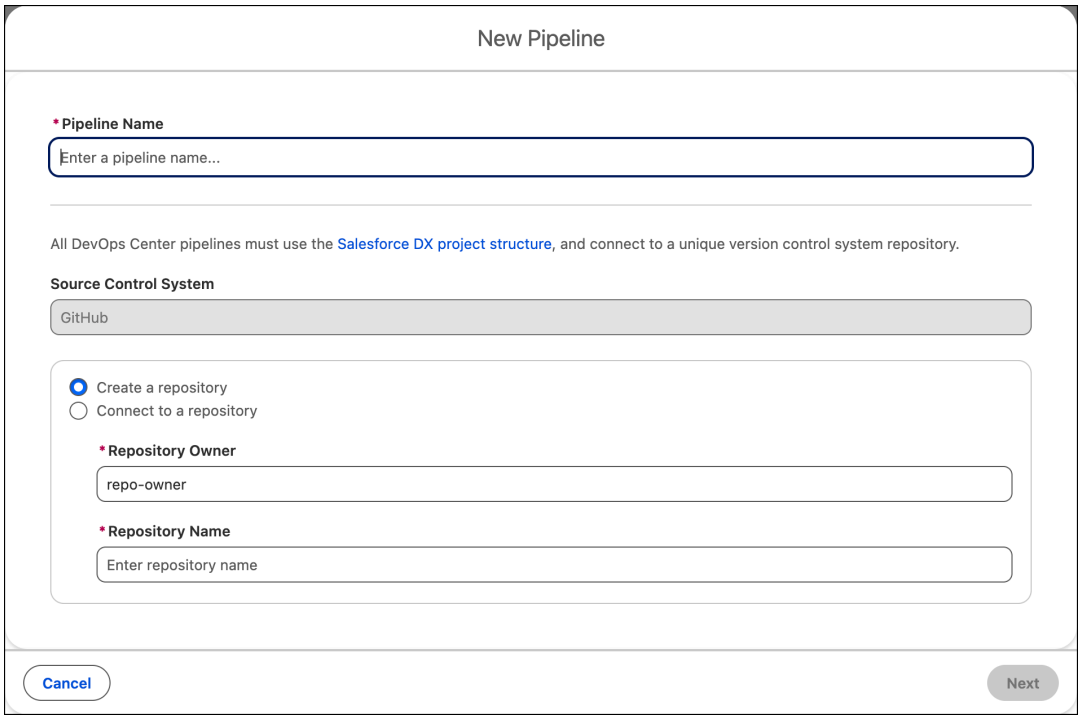
Pipeline View



Plan your pipeline carefully, as you can't modify it after you promote changes through it. The number of pipeline stages is based on the development and business processes that you have in place. If your team works in sandboxes and scratch orgs, you can connect them to the stages you plan to use for this pipeline. Besides the production and development environments, your default pipeline consists of Integration, User Acceptance Testing (UAT), and Staging stages.

1. From the DevOps Pipelines tab, click **New**.

New Pipeline Window



2. Enter a name for your pipeline.
3. Select your source control system.

   If you aren't authenticated to GitHub, you'll be prompted to log in.

4. (Optional) Choose to create a repository or add an existing one.
5. Click **Next**.
6. Select the projects to associate with this pipeline and click **Create Pipeline**.

   Your default pipeline is ready. Before you can start using the pipeline, finish setting it up.

7. For each stage in the new pipeline, complete these steps:
   a. Connect your environment.

   Note: DevOps Center pipelines require a release environment (often your production org) and one development environment. Be sure all team members are users in these environments if they require access to perform deployments from within DevOps Center.

   b. Create a branch in the source control repository for this stage or use an existing branch.
8. Click **Activate Pipeline** to turn on the pipeline.

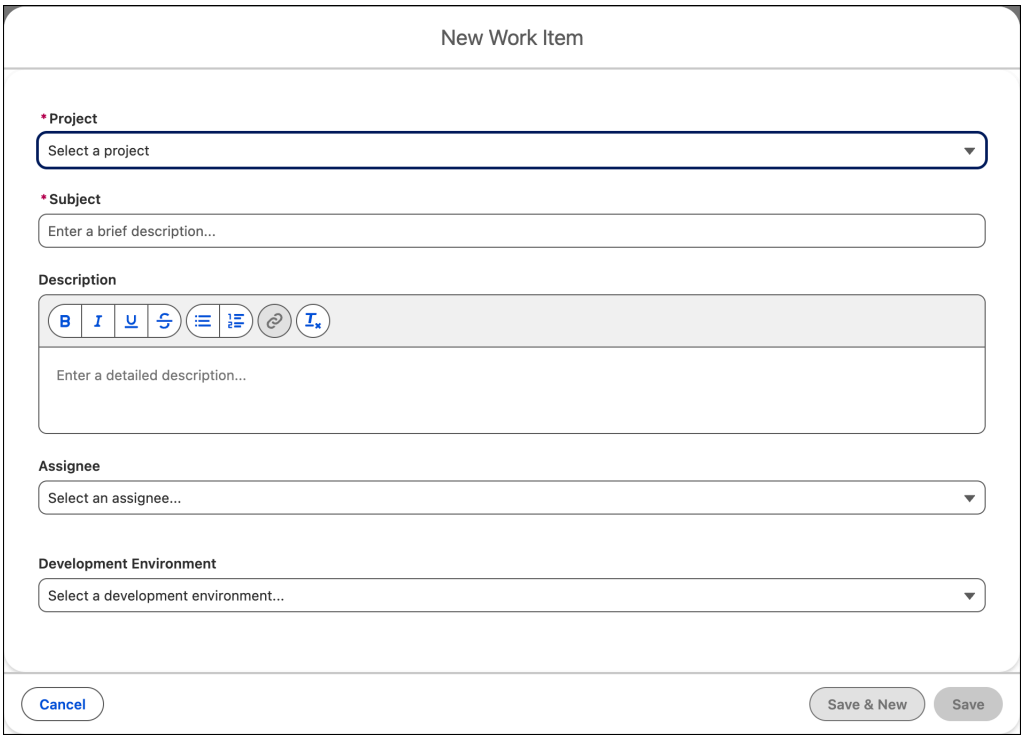
Create and Assign Work Items

Use work items to track the progress of changes related to specific objectives, such as user stories or issue fixes. Work items help teams manage changes and releases by providing visibility into the status and progress of changes. Before creating and assigning work items, create and activate your project pipeline.

1. From the Work Items tab, click **New Work Item**.

   You can also create a work item from the DevOps Projects tab. Select the project for which you're creating the work item, then click **New Work Item**.

   New Work Item Window



2. Select the DevOps Center project for which you're creating the work item.
3. In the Subject field, specify the objective or the problem.
4. In the Description field, enter additional information, including details about the changes and acceptance criteria.
   DevOps Center uses the first 255 characters of the description to identify this work item in the source control repository.
5. (Optional) Assign the work item to a team member.
6. (Optional) Select a connected development environment for this work item.

   If you don't select a development environment, the work item isn't assigned to any environment. However, when you commit this work item for the first time, the DX Inspector environment is automatically associated with it, and a branch is created. A branch is also

created when the work item moves to the `In Progress` status. You can add a development environment from the DevOps Pipelines tab.

7. Save your work item or create another.
   The work item appears in the Work Items tab, with the status New.
8. On the Work Items tab, click **Move to In Progress** to inform your team members that you're working on this work item.

Both you and your team members can create additional work items as the project progresses.

Next Steps

The basic setup for DevOps Center is complete and you can start building in your development environment. See Build Using DX Inspector.

Interested in the agentic DevOps Center experience and how you can use DevOps Center MCP tools to simplify your manual tasks? First, integrate Agentforce Vibes IDE and then set up Salesforce DX MCP Server.

Integrate Agentforce Vibes IDE with DevOps Center

Agentforce Vibes IDE provides a modern developer experience and comes with everything you need to work with Salesforce DX MCP Server and DevOps Center MCP tools.

Your trial org includes Agentforce Vibes IDE by default. After your Salesforce admin accepts the terms and conditions, Agentforce Vibes Extension becomes available in the IDE.

1. Open Agentforce Vibes IDE from your App Launcher.
2. In Agentforce Vibes IDE, click the hamburger icon ( ) in the Activity Bar, and then select **Terminal** > **New Terminal**.
3. Go to the **/home.codebuilder/directory**.
4. Clone your DevOps Center pipeline GitHub repository into the Agentforce Vibes IDE environment. See Create a Project from a Repository.

```
None
git clone https://github.com/<DevOps_Center_pipeline_repo_url>.git
```

5. Enter your GitHub credentials when prompted.
6. Depending on the type of your GitHub repository, authorize and authenticate to GitHub. See About Authentication to GitHub.
7. To switch the project to the new repository, click the hamburger icon in the Activity Bar and then click **File** > **Open Folder**.
8. Connect and authorize a Salesforce Org.

```
None
        sf org login web --set-default-dev-hub --alias DevHub
```

9. Enter your Salesforce org credentials in the browser that opens.
10. Open Agentforce Vibes Extension and accept the terms and conditions.

Get Started with Salesforce DX MCP Server

Set up Salesforce DX MCP Server (Beta) for use with Agentforce Vibes IDE, integrated with Agentforce Vibes Extension as the MCP client. Use prompts in Agentforce, a part of Agentforce Vibes Extension, to do common DevOps tasks, like managing work items, and resolving merge conflicts.

Although this example uses Agentforce Vibes IDE and Agentforce, you can configure other clients, such as Cursor or Claude Desktop. To use Visual Studio Code (VS Code) as the MCP client, see Quick Start Using the VS Code With Copilot MCP Client (Beta).

Before starting, make sure that you have these set up.

- A DevOps Center project and pipeline.
- An MCP-enabled IDE such as Agentforce Vibes IDE or VS Code.
- A user-facing MCP client, such as the Agentforce Vibes Extension. The Salesforce DX MCP Server is preconfigured with Agentforce Vibes Extension. See Build with Agentforce Vibes Extension.

Let's get started!

1. Open your IDE workspace with the DevOps Center project repository.
2. Open Agentforce from the activity bar.

   By default, Agentforce runs in Act mode. Use the toggle button to switch to Plan and Act mode.

3. Enable Salesforce DX MCP Server. See Access and Configure Agentforce Vibes Extension.
4. In the chat window, enter a prompt describing a DevOps task you want to perform. For examples of what you can do, see Use Cases.

   When you enter a prompt, Salesforce DX MCP Server determines the appropriate DevOps Center MCP tool to run. It then shows it to you along with other information.

5. Click **Continue** to run the tool and see the results of your request.

All the required setup for your ALM journey is complete. Start building your app using DX Inspector.

Build Using DX Inspector

Start working on your apps or customizations in your development org. DX Inspector is a management tool that provides quick access to metadata change tracking and visualization. It appears at the top of a page or builder within your Salesforce sandbox, scratch org, or Developer Edition (DE) org.

DX Inspector includes the name of the org (if available) that you're currently working in and provides an entry point to a panel with more metadata tracking capabilities. Before you start using these capabilities, review and accept the opt-in terms.

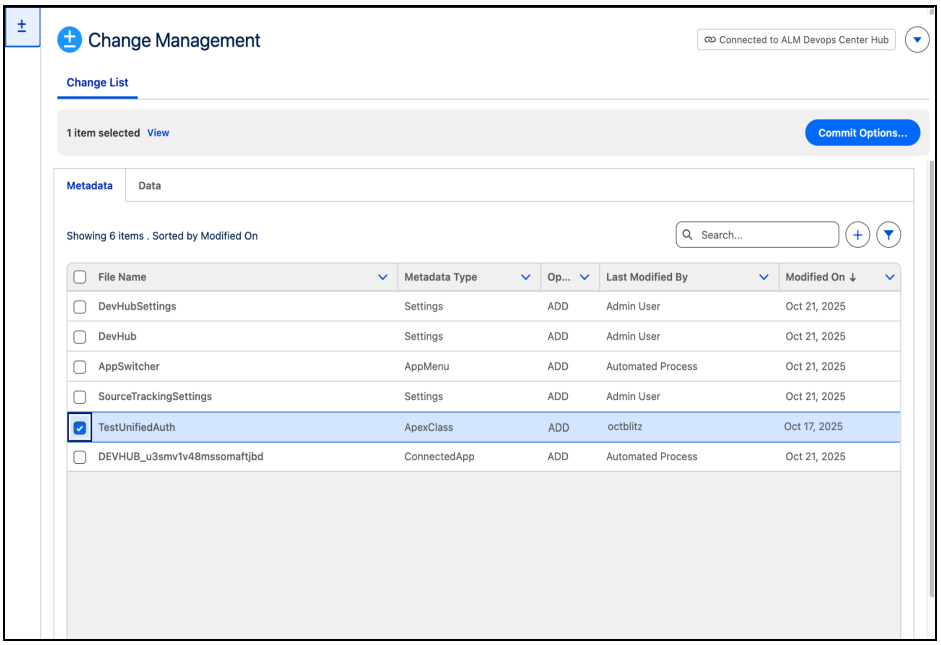
DX Inspector

Manage Changes

The Change Management page offers a unified view of all the modifications across the org you're in. From this page, you can view, manage, and commit metadata changes to your source control repository. You can also move metadata and data together for direct deployment to a target org.

Access the Change Management page by clicking the icon in the DX Inspector panel.

Change Management



Source tracking automatically tracks changes to the metadata files in sandbox and scratch orgs, which you can see on the Change Management page. Make sure that source tracking is enabled in your org. If your metadata isn't tracked using source tracking, optionally, you can manually add metadata to this tab.

To filter and view uncommitted changes, authorize and connect to DevOps Center. This filtered view shows changes since your last commit to DevOps Center. You can sort, search, and filter the list by date, org, type of metadata, manual metadata, and uncommitted changes. After you review the changes, commit the changes directly to the source control repository.

If you're not connected to DevOps Center, you can deploy metadata and data together in a single orchestration to a target org. This type of deployment is suitable for solo Salesforce admins or smaller teams. See Deploy Metadata and Data from DX Inspector.

Add Metadata Manually

The Change Management page uses source tracking to automatically track metadata changes in your org. However, when you require flexibility to include more metadata than what source tracking pulls automatically, you can add and commit additional metadata files.

USER PERMISSIONS

To add metadata manually: Customize Application

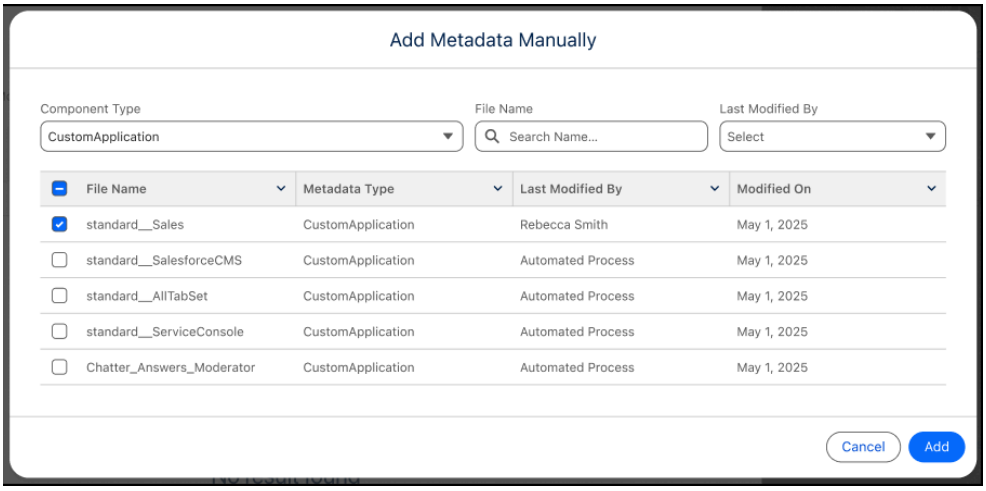
Consider adding metadata manually for these cases.

- Source tracking doesn't support the metadata file, so it isn't listed on the Change Management page. See Metadata Coverage for the list of metadata types supported by the Metadata API and source tracking.
- Your development environment is shared, and another team member already committed a file that you also want to include in your work item commit. If the metadata remains unchanged since the other team member's commit, it doesn't appear in your changes list automatically.
- You want to commit files that were changed before source tracking was enabled in your environment.
- You're in a DE org, Partial, or Full Copy sandbox.

You can't manually add metadata that's already in the Change List, and the operation type continues to reflect the original operation type (`CHANGE` or `ADD`).

1. From the DX Inspector panel, open the Change Management page.
2. Click the **Add** icon (+).
3. Select a metadata type from the dropdown. Search by file name or last modified by if you need to narrow your search results.
4. Select the metadata files you want to add.

Add Metadata Manually Window



5. Click **Add**.

The file appears on the Change Management page with `MANUAL` operation type.

Commit Changes to Source Repository

Commit changes to your source repository directly from the Change Management page. After you review the changes and decide which changes to include for the work item, commit them to the DevOps Center project's source control repository.

USER PERMISSIONS

To commit changes: Customize Application

1. Log in to your sandbox or scratch org.
2. Click on DX Inspector to access the DX Inspector panel.
3. From the DX Inspector panel, open the Change Management page.
4. Click the down arrow icon( ) and then select Connect DevOps Center.
   This action authenticates the connection to the DevOps Center GitHub source repository. On successful authentication, all uncommitted changes are listed.
5. (Optional) Add metadata manually. See Add Metadata Manually.
6. Review and select the changes you want to include for the work item. See Work Item Management.
7. Click **Commit Changes...**.
8. Select a DevOps Center project.
   All associated New, `In Progress`, `In Review`, and `Ready to Promote` work items are displayed. If there are no work items, we recommend that you go to DevOps Center and create a new work item.
9. Select a work item, and then click **Next**.

10. Add a commit comment to include details of the changes.
    This helps in identifying your commit, as there can be multiple commits to the work item.
11. Review your changes and click **Commit**.

Create a Change Review

When changes for an `In Progress` work item are committed and ready to share with the project collaborators, you can start a review using the **Create Review** button on DX Inspector. Creating a review moves the work item to `In Review` status and opens a change request, also called a pull request.

USER PERMISSIONS

To create a change review: Customize Application

A pull request is a proposal to merge the proposed changes with the rest of the code. When you start a pull request, your team members review and discuss the proposed set of changes in the source control repository.

Before you can start, commit the changes associated with your work item.

1. From DX Inspector, click **Create Review**.
   DX Inspector opens a pull request for the changes in your work item branch and updates the work item status to `In Review`.

Create Review Button on DX Inspector

2. Click **Review Created** to open the pull request in your repository.
   You're redirected to your source control repository where you or your team members can review and approve the changes.

Review and Promote Work Items in DevOps Center

Review Changes with Team Members

After you open a pull request in your source control repository, your team members and collaborators can add comments about the changes. Peer reviews help identify and fix potential issues such as bugs, coding standards violations, and performance issues.

After changes are reviewed, a team member approves the work item, and DevOps Center sets its status to `Ready to Promote`. Work items must be in this status to be available for promotion through the pipeline.

You can also manually mark a work item as ready for promotion. In DevOps Center, go to the Work Items tab, and click **Mark as Ready to Promote**.

The next step is to complete the promotion.

Promote Work Items

Move your work items through a predefined sequence of pipeline stages, guiding them from development to production. When you promote work items, their associated changes are first merged from the current stage branch to the next, and then deployed to the org for that next stage.

USER PERMISSIONS

To promote work items: Customize Application

DevOps Center supports only stage promotion (also known as work item bundle promotion in the current DevOps Center), which makes releases more stable and predictable. After a work item is considered ready to promote, it's added to the Approved Work Items list. You can promote individual, approved work items into the initial pipeline stage, where they are combined with other changes intended for the same release. Then, you promote the entire stage as a single, consistent unit to subsequent stages.

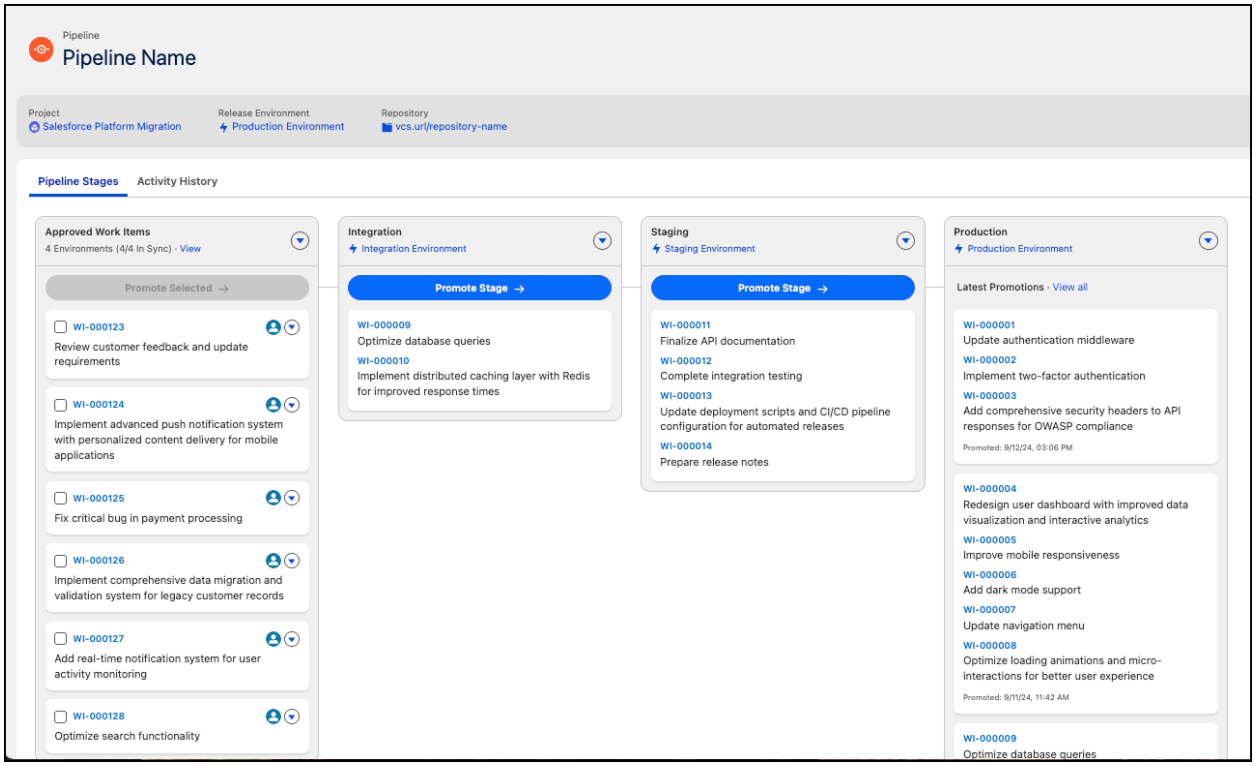
1. Go to the DevOps Pipelines tab and under Approved Work Items, select the work items to promote.
2. Click **Promote Selected**.

   The selected work items are promoted to the next stage and are now ready to be promoted as a unit.

After all work items for a release are grouped together, promote them to the next stage in the pipeline.

3. In the pipeline view, click **Promote Stage**.

Pipeline Stages



4. If you haven't logged in to the environment for the next pipeline stage from within DevOps Center, complete the login process, then click **Promote Stage**.
5. In the Promotion Options window, select which changes to promote and the Apex tests to run.
6. Click **Promote**.
   DevOps Center merge branches and deploys the work items to the environment associated with the next stage.
7. (Optional) If DevOps Center detects a merge conflict for the selected work items, it provides recommendations to resolve merge conflicts using DevOps Center MCP tools. See [Resolve Merge Conflicts].

Deploy Metadata and Data from DX Inspector

Note: Metadata deployment is a beta feature, and data deployment is a developer preview feature.

Quickly move metadata and configuration data from your current Salesforce org to a target org in a single orchestration. This type of direct deployment is ideal for moving small, specific changes without using a full DevOps pipeline.

This feature deploys metadata first and then the related data. It retrieves selected records from the source org and migrates them to the target org. To maintain data integrity, it migrates records based on their relationships. For example, the feature migrates parent records first to capture their new record IDs, and then it migrates and maps child records to the correct parent records by using these IDs. If external IDs are present, they're used to upsert records, preventing duplicates and ensuring existing records in the target org are updated with the latest changes.

Interested in learning more about this feature? See Metadata (Beta) and Data (Developer Preview) Deployment.

DevOps Center MCP Tools (Pilot)

Accelerate your application lifecycle with agentic DevOps Center MCP tools, built for DevOps Center (Pilot). Salesforce DX MCP Server provides a set of DevOps Center tools that enable Large Language Models (LLMs) to securely and autonomously read, manage, and operate DevOps Center resources. You can interact with the DevOps Center resources conversationally from Agentforce within your IDE. The MCP server acts as a context-aware backend agent that translates natural language prompts from your IDE into a sequence of API calls or CLI commands needed to perform complex DevOps tasks.

Interested in learning more about MCP servers? See MCP Solutions for Developers.

Benefits of DevOps Center MCP Tools

- Streamline manual and error-prone DevOps processes. Manage work items, create pull requests, and deploy metadata using natural language prompts.
- Reduce deployment cycle time.
- Proactively identify risks and provide AI-assisted resolution for issues such as merge conflicts.
- Implement standardized workflows and DevOps best practices across all projects and users.
- Enhance security and data retention by using encrypted authentication files, preventing data leaks, and allowing only approved access to Salesforce orgs.

Use Cases

First, set up Salesforce DX MCP Server.

Here are some sample use cases and step-by-step instructions on how you can use DevOps Center MCP tools. The provided prompts are examples and can be customized to suit your specific needs. The sample response varies based on the LLM you're using.

End-to-End Work Item Lifecycle Management

Manage your work item's lifecycle, from its initial creation and assignment to code changes, pull request creation, and promotion to different environments.

Step 1: Start with a work item

Prompt: *"Show me all open Work Items assigned to me."*
How It Works*:* The `list_devops_center_work_items` tool returns a list of work items, including their titles, branches, environments, and statuses.
Sample Response*:*

```
None
Ok. Here are your open Work Items:
```

```
Work Item: WI-1234
-Title: Bug Fix for Account Creation
-Branch:bugfix/account-create
-Environment: UAT
-Status: In Progress
```

Prompt: Let's *start working on WI-1234*
How It Works*:* The `checkout_devops_center_work_item` tool checks out the specified work item and prepares your local environment.
Sample Response:

```
None
Ok. Checking out Work Item WI-1234 and preparing your development
environment
```

Step 2: Make changes and commit them to the source repository

Prompt: "*Commit changes: Fixed validation for Contact email.*"

How It Works: The `commit_devops_center_work_item` tool verifies the current Git branch, adds the work item ID to the commit metadata, and commits the changes.
Sample Response:

```
None
Acknowledged. Commit changes with the message 'Fix validation for
Contact email'. The work item ID will be added to the commit metadata.
```

Step 3: Create a pull request

Prompt: "*Raise PR for this feature to integration.*"

How It Works: The `create_devops_center_pull_request` tool checks if commits are pushed. If not, blocks the action and prompts you to push them. If successful, creates a pull request and shows its ID and URL.

Sample Response:

```
None
Understood. Creating a pull request. Once complete, I will provide the
Pull Request ID and URL.
Here is the final output:
Pull Request ID: PR-102
Pull Request URL:
https://your-devops-platform.com/project-alpha/pull-requests/5678
```

Step 4: Review and approve pull requests

Review and approve the pull request on GitHub.

Step 5: Promote the work item

Prompt: "*Promote WI-1234 to UAT.*"

How It Works: The `promote_devops_center_work_item` tool identifies the work item ID and target environment, merges the feature branch, and deploys the metadata.

Sample Response:

```
None
Promotion of WI-1234 to UAT is initiated. You can monitor the progress
and view logs in DevOps Center.
```

Resolve Merge Conflicts

Detect, analyze, and resolve merge conflicts. You can also use the **Resolve with Agentforce** option in the DevOps Center UI to get started. This opens Agentforce Vibes IDE with Agentforce (Agentforce Vibes Extension), and the merge conflict error is added to the prompt.

Step 1: Detect and summarize conflict

Prompt: "*Check if my current branch has any merge conflicts.*"

How It Works: The `detect_devops_center_merge_conflict` tool scans Git to find the merge state. It then compares the feature branch with the target branch, identifies conflicting metadata, and suggests resolution strategies. For example, merge suggested, accept incoming, or keep yours.

Sample Response:

```
None
Conflicts have been detected between the work item branch WI-000000036
and the target branch integration. The following files have conflicts:

   -   force-app/main/default/classes/class_name.cls

       -   Conflict Type: Both branches modified the same line.

       -   Explanation: The method has been modified in both branches
      with different debug messages.

       -   Conflict Excerpt: <Details>

   -   force-app/main/default/classes/new_class_name.cls

       -   Conflict Type: Both branches modified the same line.

       -   Explanation: The method has conflicting debug messages.

       -   Conflict Excerpt: <Details>

   To proceed with resolving these conflicts, you can use the
   resolve_devops_center_merge_conflict tool for further assistance.
```

Step 2: Resolve the merge conflict

Prompt: "*Use AI to resolve the conflict*"

How It Works: The `resolve_devops_center_merge_conflict` tool loads the specified file, applies the selected resolution strategy, and stages the file.

Sample Response:

```
None

Task Completed

The merge conflicts between the work item branch WI-000000036 and the
target branch integration have been resolved successfully. The following
actions were taken:

1.  Resolved conflicts in the files:
    -   `_force-app/main/default/classes/class_name.cls_`: Kept current
changes from WI-000000036.
```

```
    -   `_force-app/main/default/classes/new_class_name.cls_`: Kept
current changes from WI-000000036.

2.  Committed the resolution with the message: "Resolve merge conflicts
between WI-000000036 and integration".

The branch is now conflict-free and ready for further actions.
```

Step 3: Resume work item promotion

Continue promoting your work items in DevOps Center.

Resolve Conflicts from the DevOps Center UI (Low-Code Admins)

If you're a low-code developer or admin, you can use DevOps Center tools from the DevOps Center UI without writing any commands.

When promoting changes through a pipeline, the deployment fails if DevOps Center detects a merge conflict. The DevOps Center UI, powered by Salesforce DX MCP Server and DevOps Center MCP tools, provides a clear, immediate notification, highlighting the specific work item that is blocked.

Use the **Resolve with Agentforce** button to resolve the merge conflict. This opens Agentforce Vibes IDE with Agentforce, and the error message is preloaded into your prompt. You can then follow the steps in Resolve Merge Conflicts to fix the issue without using CLIs or APIs.

Tool Reference

Use these DevOps Center MCP tools to interact with the DevOps Center resources:

Work Item

Tool	Description	Sample Prompts
list_devops_center_work_items	List all the work items for a specific DevOps Center project.	• *"List my Work Items for this sprint along with their branches."* • *"Give me a list of active Work Items with their environments and target branches."* • *"Fetch all the Work Items I'm assigned to and show their promotion paths."*

list_devops_center_projects	List all DevOps Center projects in a specific org.	• *"Can you list out all the projects in DevOps Center for me?"* • *"What are the DevOps Center projects currently in this org?"* • *"I'd like to see all the active DevOps Center projects."*
checkout_devops_center_work_item	Clone and open a DevOps Center work item's branch in your local development environment.	• *"Check out my WI-1234 and prepare the environment"* • *"Start work on WI-1234. Open the branch and show me the details."* • *"Switch to the branch for WI-1234 and display the associated Work Item description."*
commit_devops_center_work_item	Commit local changes to a DevOps Center work item's feature branch.	• *"Commit my changes with message 'Fix bug in account logic' and tie it to WI-1092."* • *"Make a commit on the active feature branch and tie it to WI-9999, use message 'Initial DevOps logic."* • *"Commit my changes to the work item."*
check_devops_center_commit_status	Check the current status of a work item committed to DevOps Center.	• *"What's the status of the commit for work item WI-001?"* • *"Can you check the commit request status for the latest work item?"* • *"Is the commit for work item #456 still pending?"*

Pull Request

Tool	Description	Sample Prompts
create_devops_center_pull_request	Create a pull request from a DevOps Center work item's feature branch.	• *"Create a pull request for Work Item and merge it into integration."* • *"Open a PR from my current feature branch to the integration branch."* • *"Create and register a PR to DevOps Center from my latest pushed commits."*

Pipeline and Deployment

Tool	Description	Sample Prompts
promote_devops_center_work_item	Promote an approved work item to the next stage in the DevOps Center pipeline.	• *"Deploy my current feature branch to the UAT sandbox."* • *"Promote the approved work item WI-1234 to the UAT environment."* • *"Release WI-1234 to UAT."*

Conflict Resolution

Tool	Description	Sample Prompts
detect_devops_center_merge_conflict	Identify merge conflicts between a DevOps Center work item's feature branch and its target branch.	• *"Analyze the merge conflict for my current branch against the dev branch."* • *"Help me understand the conflict that's preventing WI-xx from being promoted?"* • *"Explain why my feature branch has a conflict?"*
resolve_devops_center_merge_conflict	Apply a selected resolution method to a merge conflict.	• *"Resolve the conflict in profile-meta.yml by merging both changes."* • *"Resolve the conflict by keeping current changes."* • *"Accept all incoming changes for all conflicting files."*

Glossary

Get familiar with the key terms and concepts related to Salesforce ALM.

- **Commit**: Saving metadata changes from a development org to the project's source control repository.
- **DevOps Center Project**: A collaborative hub for teams to manage change, development, and track requirements through the ALM process.
- **Development Environment**: The development environment is where you're going to perform development and configuration tasks. It can be a Developer sandbox, Developer Pro sandbox, or scratch org.
- **Model Context Protocol (MCP):** MCP is an open standard that will allow AI agents to obtain resources, tools, and prompts from internal and external sources through a single, well-defined handshake.
- **MCP Client**: The interface (such as Agentforce) or IDE (such as Cursor) that can host an MCP server and act as an interface to the LLM.
- **MCP Server**: An MCP server lets users interact with a system (such as Salesforce or PayPal) using an LLM, and natural language instead of an API. MCP servers provide the LLM with tools, prompts, and resources that the LLM can use to perform specific tasks.
- **MCP Tools**: Executable functions that the LLM can call to perform actions.
- **Pipeline**: Defines stages for changes from development to production, syncing Salesforce orgs and branches. Each stage includes a Salesforce org and a Branch in source control.
- **Resources**: Data feeds, files, or other information that the servers can access. The LLM decides which resources to access based on the users' commands and the tool's documentation.
- **Review**: The process of examining proposed changes (often in a pull request) by team members or collaborators before they are merged into the main codebase.
- **Stage**: A distinct phase within a deployment pipeline, representing a specific environment (for example, Integration, UAT, Staging, Production) through which changes progress during the release lifecycle.
- **Source Control**: Integrated at the pipeline level for managing code and metadata versioning. Facilitates collaboration and maintains a history of changes as a single source of truth.
- **Work Item**: Represents a unit of work aligned with a business requirement, tracking the "why" behind changes.

Limitations

- You can connect to only one DevOps Center Hub org from DX Inspector. To connect to a new org, log out of the current org and delete your named credentials or external credentials.
- When you use the **Resolve with Agentforce** button to fix the merge conflict, then refresh or reopen Agentforce Vibes IDE, the merge conflict error prompt reruns automatically.

Tips for Writing Prompts for Salesforce DX MCP Server

When writing a prompt in Agentforce to solve DevOps tasks, consider these questions:

- Audience: Who is the intended user?
- Functionality: What is the task or purpose?
- Context: Does the prompt contain all the required information and is the situation clear?

The more information that you include, the more easily Salesforce DX MCP Server can determine whether the MCP tool fits your needs.

- Clearly state what you want to achieve. Instead of just naming an object, describe the action you want to perform on it.
  - Poor: "*work item WI-00501*"
  - Good: "*Show me the details for work item WI-00501*"

- Provide the necessary context, as Salesforce DX MCP Server doesn't know what you're currently working on. Always include key identifiers like work item numbers, branch names, or pipeline environments in your prompt.
  - Poor: "*Promote my changes.*"
  - Good: "*Promote work item WI-00501 to the UAT environment.*"

- Focus on a single, clear task for each prompt. If you have a multi-step process, send each instruction one at a time. This avoids confusion and gives you a chance to verify each step.
  - Poor: *“Create a new feature branch for WI-0042, commit my latest changes, then raise a pull request to integration and promote it to UAT.”*
  - Good: *“Create a new feature branch for WI-0042.”*

- Use DevOps Center as a keyword. This gives context to the Salesforce DX MCP Server and helps it select the right MCP tool for your task.
  - Poor: *“Show me the details of work item WI-00000001"*
  - Good: *“Show me the details of DevOps Center work item WI-00000001"*